PROJECT SYNOPSIS
“AGILEGRAPH”

Graph-Learned Crypto-Agility Risk Scoring for Post-Quantum Migration
(Heterogeneous GNN Risk Propagation and Explainable Migration Prioritization)

Students Names & USN:
1. Spoorthi — NNM23CB058
2. Sindhoora R — NNM23CB057
3. Nivedita V Shetty — NNM23CB033
4. Sujan S — NNM23CB062

Guide Name: Ms. Madhushree
Department: Computer Science & Engineering (Cyber Security)
Institution: NMAM Institute of Technology (NMAMIT), Nitte
Academic Year: 2023 – 2027 (Final Year, B.Tech)

1. Introduction

The advent of Cryptanalytically Relevant Quantum Computers (CRQCs) presents an existential threat to modern asymmetric cryptography. Shor’s algorithm guarantees that quantum computing operating in polynomial time can compromise public-key cryptosystems based on integer factorization (RSA) and discrete logarithms over finite fields and elliptic curves (DSA, ECDSA, ECDH). In response, the National Institute of Standards and Technology (NIST) finalized the primary post-quantum cryptographic standards in August 2024: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA).

However, transitioning modern polyglot enterprise software architectures to quantum-resistant primitives is an arduous systems engineering challenge. Organizations face the critical threat of **Harvest Now, Decrypt Later (HNDL)** attacks, where adversarial entities intercept and store encrypted sensitive data today to decrypt once quantum machines emerge. Because modern software is deeply modular and dependent on third-party supply chains, cryptographic primitives are dispersed across application source code, containerized endpoints, external package manifests, digital certificates, and continuous integration pipelines.

AgileGraph is proposed as an automated cryptographic discovery, typed property-graph modeling, and crypto-agility risk scoring system. Rather than treating codebases as flat lists of dependencies, AgileGraph structures polyglot source code (Python, Java, Go), dependency manifests, and infrastructure endpoints into a formal **6-node typed property graph**. It evaluates migration urgency through Michele Mosca’s Theorem ($U = (x+y)/z$), computes deterministic 7-factor heuristic risk scores with missing-context handling, tracks asset remediation across a 6-state lifecycle, and investigates a Heterogeneous Graph Attention Network (GATv2) for topological risk propagation and representation distillation.

2. Problem Statement

Current software supply chain and cryptographic inventory approaches suffer from four fundamental limitations:
1. Flat, Disconnected Inventories: Conventional Software Bill of Materials (SBOM) and Software Composition Analysis (SCA) tools treat software as flat dependency manifests, failing to capture architectural call paths, data flow exposure, or the structural centrality of cryptographic primitives.
2. Ignorance of Post-Quantum Urgency: Standard vulnerability scanners evaluate severity based on historical Common Vulnerability Scoring System (CVSS) metrics rather than forward-looking quantum threat timelines and data confidentiality retention requirements (Mosca's Theorem).
3. Rigid, Monolithic Scoring Without Context Handling: Existing rule engines fail when operational context (e.g., internet exposure or dynamic data classification) cannot be statically observed, either defaulting to zero risk (dangerous underestimation) or failing completely.
4. Lack of Lifecycle Remediation Tracking: Security scanners report point-in-time vulnerability lists without supporting the multi-year, phased cryptographic agility migration lifecycle (from discovery through dual-mode hybrid testing to full algorithm decommissioning).

3. Objectives

1. **Comprehensive 7-Scanner Discovery Suite:** To build a deterministic static discovery engine comprising 7 specialized scanners across 3 programming languages (Python AST, Java JCA/JCE, Go) and 4 structural/environmental analyzers (Dependency Manifests with OSV CVE intelligence, X.509 Certificate & Key parser, Network Endpoint scanner, and Sensitive Data/Secret scanner).
2. **Formal Typed Property Graph Modeling:** To design and construct an in-memory 6-node typed property graph (*File*, *CryptoUsage*, *Certificate*, *Endpoint*, *Library*, *SensitiveData*) interconnected by 7 canonical directed relationships.
3. **Deterministic 7-Factor Heuristic Risk Engine:** To implement a normalized composite risk formula evaluating algorithmic weakness, asset criticality, data sensitivity, internet exposure, migration difficulty, library centrality, and known CVE risks, featuring configurable missing-evidence policies (*RENORMALIZE* and *CONSERVATIVE*).
4. **Mosca Urgency & 4-Tier Triage Matrix:** To quantify post-quantum migration urgency ($U = (x+y)/z$) against quantum threat horizons and assign cryptographic assets into actionable triage bands (*CRITICAL*, *HIGH*, *MEDIUM*, *LOW*).
5. **Heterogeneous Graph Neural Network (GATv2) Integration:** To evaluate a PyG Heterogeneous GATv2 model to explore graph representation learning and structural risk propagation across code property graphs under Leave-One-Project-Out (LOPO) cross-validation.
6. **6-State Remediation Lifecycle & Interactive Dashboard:** To deliver an enterprise-grade React/TypeScript web dashboard and REST API backed by an append-only audit trail tracking assets across 6 lifecycle states.

4. Literature Survey

4.1 Existing System

Existing enterprise crypto-inventory mechanisms typically rely on basic static analysis (SAST) linters (e.g., Bandit for Python, SpotBugs/FindSecBugs for Java) or commercial SCA tools (e.g., Snyk, Mend). These tools execute regex-based or token-based searches to flag deprecated algorithm names (such as MD5, SHA-1, or DES). Recently, cryptographic bill of materials (CBOM) generators have emerged to catalog cryptographic libraries declared in software build manifests.

4.2 Limitations of Existing System

Existing systems fail to model the relational architecture connecting code files to endpoints and libraries. A deprecated cipher buried in an offline test suite is given the same severity rating as the same cipher securing a public API endpoint. Furthermore, none of the current tools incorporate Michele Mosca's post-quantum urgency timeline, making it impossible for engineering teams to prioritize migration based on data retention requirements.

4.3 Related Research Papers

Sl.	Paper Title & Venue	Authors & Year	Key Contribution	Gap Addressed by AgileGraph
1	Transitioning to Post-Quantum Cryptography (IEEE S&P;)	M. Mosca (2022)	Formulated Mosca’s theorem (x+y > z) defining quantum migration deadlines.	Lacks an automated static analysis engine to extract x, y, and code-level parameters.
2	SoK: Cryptographic Agility in Practice (EuroS&P;)	K. A. et al. (2023)	Analyzed enterprise agility bottlenecks and defined crypto-agility dimensions.	Provided theoretical taxonomies without graph-based static vulnerability modeling.
3	How Do Open-Source Projects Migrate to PQC? (ACM TOSEM)	X. Chen et al. (2024)	Empirical audit of PQC migration patterns and hybrid cryptosystem adoption.	Highlighted developer errors in PQC migration; lacked automated risk scoring tools.
4	How Attentive Are Graph Attention Networks? (ICLR)	S. Brody et al. (2022)	Proved dynamic attention limitations in standard GAT and introduced GATv2.	GATv2 evaluated only on citation graphs, not on heterogeneous code security graphs.
5	Automated Cryptographic Discovery in ASTs (Comp & Sec)	R. Kumar et al. (2023)	Developed AST pattern matching for cryptographic primitive identification.	Restricted to single-language codebases without multi-layer property graph construction.

5. Proposed System

The proposed AgileGraph architecture operates through five cohesive technical layers:

1. Comprehensive 7-Scanner Discovery Suite: Executes 7 specialized static scanners mapping directly into the 6 property-graph node categories: (a) Python AST Scanner (hashlib, hmac, cryptography.hazmat, PyCryptodome), (b) Java Scanner (JCA/JCE Cipher, KeyPairGenerator, jjwt, java-jwt), (c) Go Scanner (crypto/*, golang-jwt), (d) Dependency Manifest Scanner (requirements.txt, pom.xml, go.mod with local OSV CVE cache), (e) Certificate & Key Scanner (X.509 RFC 5280 parsing, inline PEM blocks, private keys), (f) Network Endpoint Scanner (service bindings, port configurations, route decorators), and (g) Sensitive Data Scanner (Shannon entropy calculation, secret tokens, private credentials).

2. 6-Node Typed Property Graph: Models software architecture in NetworkX across *File*, *CryptoUsage*, *Certificate*, *Endpoint*, *Library*, and *SensitiveData* nodes interconnected by 7 canonical relations (*CONTAINS*, *IMPORTS*, *DEPENDS_ON*, *CALLS*, *EXPOSES*, *PROTECTS*, *USES_CERT*).

3. Deterministic 7-Factor Risk Engine: Computes normalized risk scores based on algorithmic weakness (NIST PQC status), asset criticality, data sensitivity, internet exposure, migration difficulty, library centrality, and CVE severity, with robust missing-evidence handling (*RENORMALIZE* and *CONSERVATIVE*).

4. Mosca Urgency & Triage Matrix: Computes $U = (x+y)/z$ to categorize assets into *AT_RISK*, *URGENT*, and *SAFE*, triaging vulnerabilities into *CRITICAL*, *HIGH*, *MEDIUM*, and *LOW* priority bands.

5. Heterogeneous GATv2 Neural Network: Implements a PyG HeteroGATv2 model to evaluate structural representation learning and cross-codebase risk distillation under Leave-One-Project-Out cross-validation.

6. Presentation & Lifecycle: An interactive React 19/TypeScript web dashboard featuring 2D force-directed graph rendering, Kanban remediation tracking across 6 states, and SHA-256 verifiable audit report export.

6. System Requirements

6.1 Software Requirements

• **Operating System:** Ubuntu Linux 22.04 LTS / Windows 11 / macOS Sonoma
• **Programming Languages:** Python 3.10+ (Backend & GNN), TypeScript 5.0+ (Frontend)
• **Frameworks:** FastAPI, Uvicorn, Pydantic v2, React 19, Vite, TailwindCSS v4
• **Libraries:** NetworkX 3.2+, PyTorch 2.4+, PyTorch Geometric (PyG) 2.6+, Scikit-Learn, SciPy
• **Database:** SQLite (WAL mode for session security and append-only audit trail)
• **Security:** bcrypt (12 rounds) + PyJWT (HS256) bearer authentication
• **Packaging:** Docker & Docker Compose

6.2 Hardware Requirements

• **Processor:** Quad-Core Intel Core i5 / AMD Ryzen 5 or higher (Apple Silicon supported)
• **RAM:** 8 GB minimum (16 GB recommended for GATv2 model training)
• **Storage:** 10 GB available SSD space

7. Expected Outcomes

1. **Automated Polyglot 7-Scanner Suite:** A deterministic CLI and background worker executing 7 specialized scanners across Python, Java, Go, package manifests, certificates, endpoints, and secrets.
2. **Interactive Web Dashboard:** A full-stack application featuring 2D force-directed graph visualization, inventory ledgers, and Mosca calculators.
3. **Kanban Remediation Workflow:** Lifecycle tracking enabling engineering teams to transition assets from DISCOVERED to DECOMMISSIONED with immutable SQLite audit logs.
4. **Reproducible LOPO Benchmark Study:** Empirical cross-validation across 8 benchmark repositories (520 files) documenting representation distillation.
5. **Verifiable Audit Exporters:** Export of compliance documentation in JSON and Markdown formats with SHA-256 provenance hashes.

8. Applications

- **Enterprise Post-Quantum Migration:** Enables CISOs and security architects to identify and prioritize high-risk cryptographic dependencies before quantum hardware scales.
- **DevSecOps & CI/CD Gating:** Integrates into continuous deployment pipelines to prevent developers from introducing non-agile, quantum-vulnerable cryptographic algorithms into production.
- **Cybersecurity Compliance Auditing:** Automatically generates verifiable cryptographic compliance documentation aligned with NIST FIPS 203/204/205, BSI post-quantum roadmaps, and NSA CNSA 2.0 timelines.
- **Software Supply Chain Risk Management:** Evaluates open-source libraries and proprietary vendor packages to assess transitive cryptographic exposure and structural library centrality.

9. Future Scope

- **Automated Code Patching via LLMs:** Integrating fine-tuned Large Language Models to automatically generate code pull requests that refactor deprecated cryptographic calls into quantum-safe primitives.

- **Runtime Passive eBPF Agent:** Expanding discovery from static code analysis to kernel-level eBPF probes that monitor live TLS handshakes, dynamic certificate rotations, and active socket connections in production Kubernetes clusters.
- **Distributed Graph Database Migration:** Upgrading the in-memory NetworkX storage to distributed graph databases (such as Neo4j or Memgraph) to support enterprise code repositories exceeding millions of source files.

10. Project Timeline

Phase	Milestone / Activity Description	Duration	Target Deliverable
Phase 1	Requirement analysis, NIST PQC review, AST scanner development for Python, Java, Go.	Weeks 1–4	Multi-language AST discovery modules and reference test fixtures.
Phase 2	Graph schema formulation, 6-node ontology design, and NetworkX property graph builder.	Weeks 5–8	Validated property graph generator producing verified edge triplets.
Phase 3	Deterministic 7-factor risk engine, missing data policies, and Mosca theorem urgency integration.	Weeks 9–12	Verified mathematical scoring engine and JSON export utilities.
Phase 4	Benchmark corpus compilation (8 repos, 520 files), GATv2 model engineering, and LOPO cross-validation.	Weeks 13–16	Evaluated PyG GNN pipeline, multi-seed stability results, and baseline comparisons.
Phase 5	Protected React 19 web dashboard, Kanban lifecycle board, JWT authentication, and CLI reporting.	Weeks 17–20	Fully integrated full-stack web application with force-directed graph rendering.
Phase 6	System hardening, Docker packaging, comprehensive regression testing, documentation, and final viva defense.	Weeks 21–24	Final project report, research paper manuscript, and production-ready source code.

11. References

[1] National Institute of Standards and Technology (NIST), “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” Federal Information Processing Standards Publication (FIPS PUB) 203, Aug. 2024.

[2] National Institute of Standards and Technology (NIST), “Module-Lattice-Based Digital Signature Standard,” Federal Information Processing Standards Publication (FIPS PUB) 204, Aug. 2024.

[3] National Institute of Standards and Technology (NIST), “Stateless Hash-Based Digital Signature Standard,” Federal Information Processing Standards Publication (FIPS PUB) 205, Aug. 2024.

[4] M. Mosca, “Transitioning to Post-Quantum Cryptography: Challenges and Recommendations,” IEEE Security & Privacy, vol. 20, no. 5, pp. 60–66, Sep.–Oct. 2022, doi: 10.1109/MSEC.2022.3188752.

[5] S. Brody, U. Alon, and E. Yahav, “How Attentive Are Graph Attention Networks?,” in Proc. 10th Int. Conf. Learn. Represent. (ICLR), Virtual Event, Apr. 2022, pp. 1–18.

[6] X. Chen, Y. Zhang, L. Lu, and J. Grundy, “How Do Open-Source Projects Migrate to Post-Quantum Cryptography? An Empirical Study,” ACM Transactions on Software Engineering and Methodology (TOSEM), vol. 33, no. 4, pp. 1–32, Mar. 2024, doi: 10.1145/3638245.

[7] IBM Research, “Cryptographic Bill of Materials (CBOM) Specification: Towards Automated Cryptographic Observability and Agility,” IEEE Software, vol. 40, no. 3, pp. 45–52, May–Jun. 2023.

[8] K. A. Bernsmed, C. Fritsch, and M. G. Jaatun, “Security and Agility in Practice: An Empirical Survey on Cryptographic Transitions,” in Proc. 8th IEEE Eur. Symp. Secur. Privacy (EuroS&P;), Delft, Netherlands, Jul. 2023, pp. 312–326.

[9] R. Kumar, A. Singhal, and P. Liu, “Automated Discovery and Dependency Modeling of Cryptographic APIs in Polyglot Software Systems,” Computers & Security, vol. 128, p. 103140, May 2023, doi: 10.1016/j.cose.2023.103140.

[10] National Security Agency (NSA), “Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) Cybersecurity Advisory,” NSA Cybersecurity Technical Report, Sep. 2022.

Student Signatures:

1. Spoorthi (NNM23CB058): _____

2. Sindhoora R (NNM23CB057): _____

3. Nivedita V Shetty (NNM23CB033): _____

4. Sujan S (NNM23CB062): _____

Guide Signature: _____
(Ms. Madhushree)
Department of Computer Science and Engineering (Cyber Security)
NMAMIT, Nitte